

NAME:RAJA GOPAL
ROLL NUMBER:2503A51L01

TASK 1

PROMPT:UseAtohelpimplementaStackclassinPython with the following operations: push(), pop(), peek(),and is_empty()

CODE:

```
> Users > Rajalaami > Downloads > new.py > Stack > peek
1 class Stack:
2     """A simple stack implementation using a Python list.
3
4     Supports push, pop, peek, and is_empty operations.
5     """
6
7     def __init__(self):
8         """Initializes an empty stack."""
9         self._items = []
10
11     def push(self, item):
12         """Pushes an item onto the stack.
13
14         Args:
15             item: The item to be added.
16         """
17         self._items.append(item)
18
19     def pop(self):
20         """Removes and returns the top item from the stack.
21
22         Returns:
23             The item at the top of the stack.
24
25         Raises:
26             IndexError: If the stack is empty.
27         """
28         if self.is_empty():
29             raise IndexError("pop from empty stack")
30         return self._items.pop()
31
32     def peek(self):
33         """Returns the top item without removing it.
34
35         Returns:
36             The item at the top of the stack.
37
38         Raises:
39             IndexError: If the stack is empty.
40         """
41         if self.is_empty():
42             raise IndexError("peek from empty stack")
43         return self._items[-1]
44
45     def is_empty(self):
46         """Checks if the stack is empty.
47
48         Returns:
49             True if the stack is empty, False otherwise.
50         """
51         return len(self._items) == 0
52
53 # Sample usage and testing
54 if __name__ == "__main__":
55     stack = Stack()
56     stack.push(10)
57     stack.push(20)
58     stack.push(30)
59     print("Peek:", stack.peek()) # Should print 30
60     print("Pop:", stack.pop()) # Should print 30
61     print("Peek after pop:", stack.peek()) # Should print 20
62     print("Is empty?", stack.is_empty()) # Should print False
63     stack.pop()
```

```

63     stack.pop()
64     stack.pop()
65     print("Is empty after popping all?", stack.is_empty()) # Should print True
66
67     # Optimization suggestion:
68     # For faster pops from the left (if needed), use collections.deque:
69     # from collections import deque
70     # self._items = deque()
71     # This allows O(1) pops from both ends, but for a classic stack, list is

```

OUTPUT:

```

PS C:\Users\Rajalaxmi\Downloads> & 'c:\Users\Rajalaxmi\AppData\Local\Programs\Python\Python310\python.exe' 'c:\Users\Rajalaxmi\vscode\extensions\ms-python.debugpy-2025.10.0-win32-x64\bundle\libs\debugpy\launcher' '59450' '--' 'C:\Users\Rajalaxmi\Downloads\new.py'
Peek: 30
Pop: 30
Peek after pop: 20
Is empty? False
Is empty after popping all? True
PS C:\Users\Rajalaxmi\Downloads>

```

OBSERVATION:

1. The stack is implemented using a Python list with efficient `append()` and `pop()` operations.
2. Each method is clearly documented with Google-styled docstrings explaining purpose, parameters, and return values.
3. The code handles edge cases like popping from or peeking into an empty stack gracefully by returning `None`.
4. The test cases cover pushing, peeking, popping, and checking if the stack is empty at various stages.

TASK2

PROMPT: Implement a Queue with `enqueue()`, `dequeue()`, and `is_empty()` method

CODE:

```

# Queue implementation using Python list
class ListQueue:
    """A simple queue implementation using a Python list."""

    def __init__(self):
        """Initializes an empty queue."""
        self._items = []

    def enqueue(self, item):
        """Adds an item to the end of the queue.

        Args:
            item: The item to be added.
        """
        self._items.append(item)

    def dequeue(self):
        """Removes and returns the item from the front of the queue.

        Returns:
            The item at the front of the queue.

        Raises:

```

```
# Queue implementation using Python list
class ListQueue:
    """A simple queue implementation using a Python list."""

    def __init__(self):
        """Initializes an empty queue."""
        self.items = []

    def enqueue(self, item):
        """Adds an item to the end of the queue.

        Args:
            item: The item to be added.
        """
        self.items.append(item)

    def dequeue(self):
        """Removes and returns the item from the front of the queue.

        Returns:
            The item at the front of the queue.

        Raises:
            IndexError: If the queue is empty.
        """
```

```
C:\Users\Rajalaxmi> Downloads> new.py > ...
2 class ListQueue:
17     def dequeue(self):
19         """Removes and returns the item from the front of the queue.
21         Returns:
22             The item at the front of the queue.
23         Raises:
24             IndexError: If the queue is empty.
25         """
26         if self.is_empty():
27             raise IndexError("dequeue from empty queue")
28         return self.items.pop(0) # O(n) operation
29
30     def is_empty(self):
31         """Checks if the queue is empty.
32
33         Returns:
34             True if the queue is empty, False otherwise.
35         """
36         return len(self.items) == 0
37
38 # Optimized queue implementation using collections.deque
39 from collections import deque
40
41 class DequeQueue:
```

```
"""A queue implementation using collections.deque for efficiency."""

def __init__(self):
    """Initializes an empty queue."""
    self.items = deque()

def enqueue(self, item):
    """Adds an item to the end of the queue."""
    self.items.append(item)

def dequeue(self):
    """Removes and returns the item from the front of the queue."""
    if self.is_empty():
        raise IndexError("dequeue from empty queue")
    return self.items.popleft() # O(1) operation

def is_empty(self):
    """Checks if the queue is empty."""
    return len(self.items) == 0

# Performance Review:
# ListQueue: dequeue() uses pop(0), which is O(n) because all elements must shift.
# DequeQueue: dequeue() uses popleft(), which is O(1) and much faster for large queues.
# For production code, prefer collections.deque for queue implementations.
```

```
# Sample usage and testing
if __name__ == "__main__":
    print("Testing ListQueue:")
    lq = ListQueue()
    lq.enqueue(1)
    lq.enqueue(2)
    lq.enqueue(3)
    print(lq.dequeue()) # 1
    print(lq.dequeue()) # 2
    print("Is empty?", lq.is_empty()) # False

    print("\nTesting DequeQueue:")
    dq = DequeQueue()
    dq.enqueue(10)
    dq.enqueue(20)
    dq.enqueue(30)
    print(dq.dequeue()) # 10
    print(dq.dequeue()) # 20
    print("Is empty?", dq.is_empty())
```

OUTPUT:

```
python310\python.exe' 'c:\Users\Rajalaxmi\.vscode\extensions\ms-python.debugpy-2025.10.0-win32-x64\bundle\libs\debugpy\launcher
r' '51812' '...' 'C:\Users\Rajalaxmi\Downloads\new.py'
Testing ListQueue:
1
2
Is empty? False

Testing DequeQueue:
10
20
Is empty? False
PS C:\Users\Rajalaxmi\Downloads> []
```

OBSERVATION:

1. The list-based queue is simple but inefficient for frequent dequeue() operations because removing from the front requires shifting all elements.
2. The deque-based queue is optimal as it provides $O(1)$ complexity for both enqueue and dequeue.
3. For real-world applications where queues are used extensively or with many elements, collections.deque is the superior choice.
4. Both implementations handle empty queues gracefully and include clear docstrings and edge-case handling

TASK3

PROMPT: Implement a Singly Linked List with operations:
insert_at_end(), delete_value(), and traverse()

CODE:

```
C:\Users\Rajalaxmi\Downloads> new.py
1 class Node:
2     """Represents a node in a singly linked list."""
3     def __init__(self, data):
4         self.data = data
5         self.next = None # Pointer to the next node
6
7 class LinkedList:
8     """Singly linked list implementation."""
9
10    def __init__(self):
11        self.head = None # Start with an empty list
12
13    def insert_at_end(self, data):
14        """Inserts a new node with the given data at the end of the list."""
15        new_node = Node(data)
16        if not self.head:
17            self.head = new_node # If list is empty, new node becomes head
18            return
19        current = self.head
20        while current.next:
21            current = current.next # Traverse to the last node
22        current.next = new_node # Update last node's next pointer
23
24    def delete_value(self, value):
25        """Deletes the first node with the specified value."""
26        current = self.head
27        prev = None
28        while current:
29            if current.data == value:
30                if prev:
31                    # Bypass the node to be deleted
32                    prev.next = current.next
33                else:
34                    # Deleting the head node
35                    self.head = current.next
36            prev = current
37            current = current.next
38        return True # Value found and deleted
39        prev = current
40
41    def traverse(self):
42        """Traverse the list and print all nodes' data"""
43        if not self.head:
44            return
45        current = self.head
46        while current:
47            print(current.data)
48            current = current.next
```

```

    # Move both pointers forward
    current = current.next
    return False # Value not found

def traverse(self):
    """Traverse the list and returns a list of node values."""
    result = []
    current = self.head
    while current:
        result.append(current.data)
        current = current.next # Move to next node
    return result

# Suggested test cases to validate all operations
if __name__ == '__main__':
    ll = LinkedList()
    # Test Insertion
    ll.insert_at_end(10)
    ll.insert_at_end(20)
    ll.insert_at_end(30)
    print("After insertions:", ll.traverse()) # [10, 20, 30]

    # Test deletion (middle value)
    ll.delete_value(20)
    print("After deleting 20:", ll.traverse()) # [10, 30]

    # Test deletion (head value)
    ll.delete_value(10)
    print("After deleting 10:", ll.traverse()) # [30]

    # Test deletion (non-existent value)
    result = ll.delete_value(99)
    print("Delete 99 (should be False):", result) # False

    # Test deletion (last value)

```

```

61     print("After deleting 20:", ll.traverse()) # [10, 30]
62
63     # Test deletion (head value)
64     ll.delete_value(10)
65     print("After deleting 10:", ll.traverse()) # [30]
66
67     # Test deletion (non-existent value)
68     result = ll.delete_value(99)
69     print("Delete 99 (should be False):", result) # False
70
71     # Test deletion (last value)
72     ll.delete_value(30)
73     print("After deleting 30:", ll.traverse()) # []
74
75     # Test insertion after all deletions
76     ll.insert_at_end(40)
77     print("After inserting 40:", ll.traverse()) # [40]

```

OUTPUT:

```

After insertions: [10, 20, 30]
After deleting 20: [10, 30]
After deleting 10: [30]
Delete 99 (should be False): False
After deleting 30: []
After inserting 40: [40]
PS C:\Users\Rajalaxmi\Downloads>

```

OBSERVATION:

- The linkedlist correctly supports insertion at the end, deletion by value, and traversal.
- Pointer updates are handled carefully:
 - When inserting at the end, the last node's pointer is updated to link the new node.
 - When deleting, both head and non-head deletions are treated properly.
- Edge cases like deleting from an empty list or deleting non-existent elements are handled safely.
- The test cases cover normal usage and edge cases ensuring robustness.

TASK4

PROMPT:

implement a Binary Search Tree with methods for insert(), search(), and inorder_traversal()

CODE:

```
1 class Node:
2     """Represents a node in a Binary Search Tree."""
3     def __init__(self, data):
4         self.data = data
5         self.left = None # Left child
6         self.right = None # Right child
7
8 class BST:
9     """Binary Search Tree implementation."""
10
11     def __init__(self):
12         self.root = None
13
14     def insert(self, data):
15         """Inserts a value into the BST.
16
17         Args:
18             data: The value to insert.
19         """
20         def _insert(node, data):
21             if node is None:
22                 return Node(data)
23             if data < node.data:
24                 node.left = _insert(node.left, data)
25             elif data > node.data:
26                 node.right = _insert(node.right, data)
27             # If data == node.data, do not insert duplicates
28             return node
29         self.root = _insert(self.root, data)
30
31     def search(self, value):
32         """Searches for a value in the BST.
33
34         Args:
35             value: The value to search for.
36         """
```

```
37
38         Returns:
39             True if found, False otherwise.
40         """
41         def _search(node, value):
42             if node is None:
43                 return False
44             if value == node.data:
45                 return True
46             elif value < node.data:
47                 return _search(node.left, value)
48             else:
49                 return _search(node.right, value)
50         return _search(self.root, value)
51
52     def inorder_traversal(self):
53         """Performs inorder traversal and returns a list of values."""
54         result = []
55         def _inorder(node):
56             if node:
57                 _inorder(node.left)
58                 result.append(node.data)
59                 _inorder(node.right)
60         _inorder(self.root)
61         return result
62
63 # Test cases
64 if __name__ == "__main__":
65     bst = BST()
66     nums = [50, 30, 70, 20, 40, 60, 80]
67     for num in nums:
68         bst.insert(num)
69
70     print("Inorder Traversal:", bst.inorder_traversal()) # [20, 30, 40, 50, 60, 70, 80]
71     print("Search 40:", bst.search(40)) # True
```

OUTPUT:

```
Inorder Traversal: [20, 30, 40, 50, 60, 70, 80]
Inorder Traversal: [20, 30, 40, 50, 60, 70, 80]
Search 40: True
Search 40: True
Search 100: False
PS C:\Users\Rajalaxmi\Downloads> █
```

OBSERAVTION:

1. Correctness

- TheBSTcorrectly insertselementsmaintainingthebinarysearchproperty.
- Inordertraversalproducesasortedlistofinserted elements.
- Thesearchmethodcorrectlyidentifieswhetherelementsarepresentor absent.

2. Edgecases handled

- Searchingforelementsnotinthetreereturns False.
- SearchingforelementsinthetreereturnsTrue.

3. Designclarity

- Therecursiveapproachinbothinsertionandsearchensurescleanand concise logic.
- Thehelperfunctionsclearlyseparate recursionfromthepublicAPImethods.

TASK5

PROMPT:ImplementaGraphusinganadjacencylist, withtraversal methods BFS() and DFS().

CODE:

```
class Graph:
    """Graph represented using an adjacency list."""
    def __init__(self):
        self.adj_list = {} # Dictionary to store adjacency list
    def add_edge(self, u, v):
        """Adds an edge from u to v (undirected)."""
        if u not in self.adj_list:
            self.adj_list[u] = []
        if v not in self.adj_list:
            self.adj_list[v] = []
        self.adj_list[u].append(v)
        self.adj_list[v].append(u)
    def bfs(self, start):
        """Performs Breadth-First Search (BFS) traversal from the start node.
        Returns:
            List of nodes in BFS order.
        """
        visited = set()
        queue = [start]
        order = []
        while queue:
            node = queue.pop(0) # Dequeue the front node
            if node not in visited:
                visited.add(node)
                order.append(node)
                # Enqueue all unvisited neighbors
                for neighbor in self.adj_list.get(node, []):
                    if neighbor not in visited:
                        queue.append(neighbor)
        return order
    def dfs_recursive(self, start):
```

```

def dfs_recursive(self, start):
    """Performs Depth-First Search (DFS) recursively from the start node.

    Returns:
    List of nodes in DFS order.
    """
    visited = set()
    order = []

    def _dfs(node):
        if node not in visited:
            visited.add(node)
            order.append(node)
            # Recursively visit all neighbors
            for neighbor in self.adj_list.get(node, []):
                _dfs(neighbor)

    _dfs(start)
    return order

def dfs_iterative(self, start):
    """Performs Depth-First Search (DFS) iteratively from the start node.

    Returns:
    List of nodes in DFS order.
    """
    visited = set()
    stack = [start]
    order = []

    while stack:
        node = stack.pop() # Pop the top node
        if node not in visited:
            visited.add(node)
            order.append(node)
            # Push all unvisited neighbors to stack
            for neighbor in reversed(self.adj_list.get(node, [])):
                if neighbor not in visited:
                    stack.append(neighbor)

    return order

# Example usage and comparison
if __name__ == "__main__":
    g = Graph()
    g.add_edge('A', 'B')
    g.add_edge('A', 'C')
    g.add_edge('B', 'D')
    g.add_edge('C', 'E')
    g.add_edge('D', 'E')
    g.add_edge('E', 'F')

    print("Adjacency List:", g.adj_list)
    print("BFS from A:", g.bfs('A')) # ['A', 'B', 'C', 'D', 'E', 'F']
    print("Recursive DFS from A:", g.dfs_recursive('A')) # DFS order
    print("Iterative DFS from A:", g.dfs_iterative('A')) # DFS order

    # Note: Recursive DFS uses function calls, iterative DFS uses a stack.
    # Both visit all reachable nodes, but order may differ depending on adjacency list.

```

OUTPUT:

```

Adjacency List: {'A': ['B', 'C'], 'B': ['A', 'D'], 'C': ['A', 'E'], 'D': ['B', 'E'], 'E': ['C', 'D', 'F'], 'F': ['E']}
BFS from A: ['A', 'B', 'C', 'D', 'E', 'F']
Recursive DFS from A: ['A', 'B', 'D', 'E', 'C', 'F']
Iterative DFS from A: ['A', 'B', 'D', 'E', 'C', 'F']
PS C:\Users\Rajalaxmi\Downloads>

```

OBSERVATION:

1. Graph Representation:

- The adjacency list is efficient in representing sparse graphs.
- Neighbors are stored in lists for each node.

2. BFS Traversal:

- Uses a queue (FIFO structure).
- Visits all neighbors level by level.
- Suitable for shortest path problems in unweighted graphs.

DFS TRAVERSAL:

- Can be implemented both recursively and iteratively.
- Recursive DFS is simpler to write and more elegant.
- Iterative DFS avoids recursion depth issues and is more robust for large graphs.